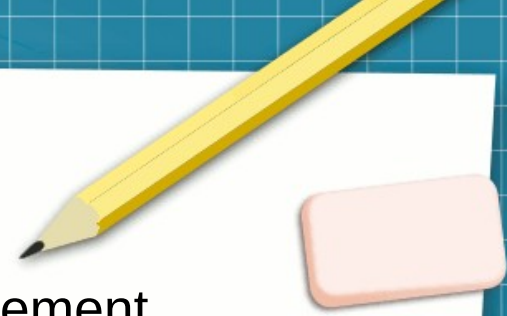




Residential Smart Grid

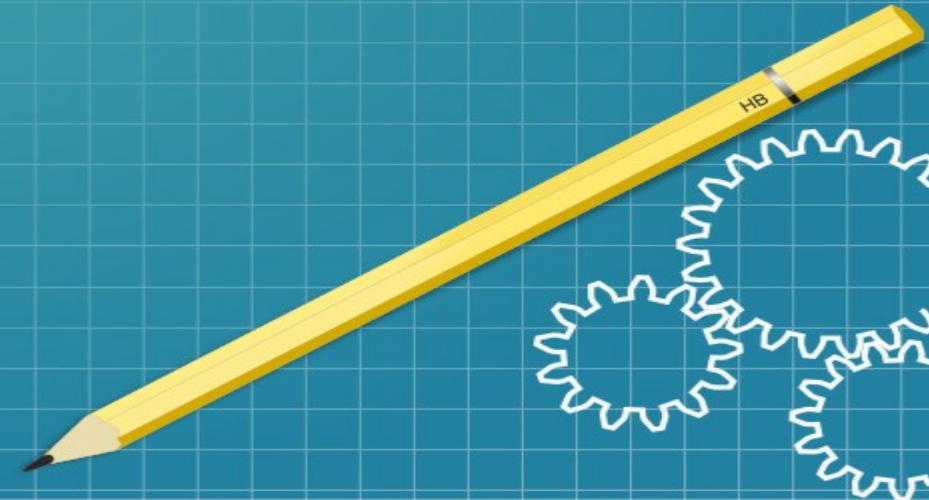
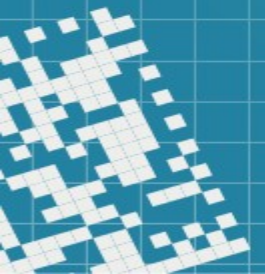
Report - 01

Table of Contents



- Problem Statement
- Project Overview
 - Project Description
 - Project Features
 - System Design
 - Prototype System Design
- Current Progress & Plans
 - Software Simulation
 - Hardware Houses Controls
 - Power Management
 - Householder App APIs
 - Householder Mobile App
 - Admin Dashboard
- Notebooks
 - Introduction to Notebooks
 - Logs Notebook
 - Graphs Notebook
 - PV Notebook
- Documentation & Dissertation

Problem Statement



Problem Statement



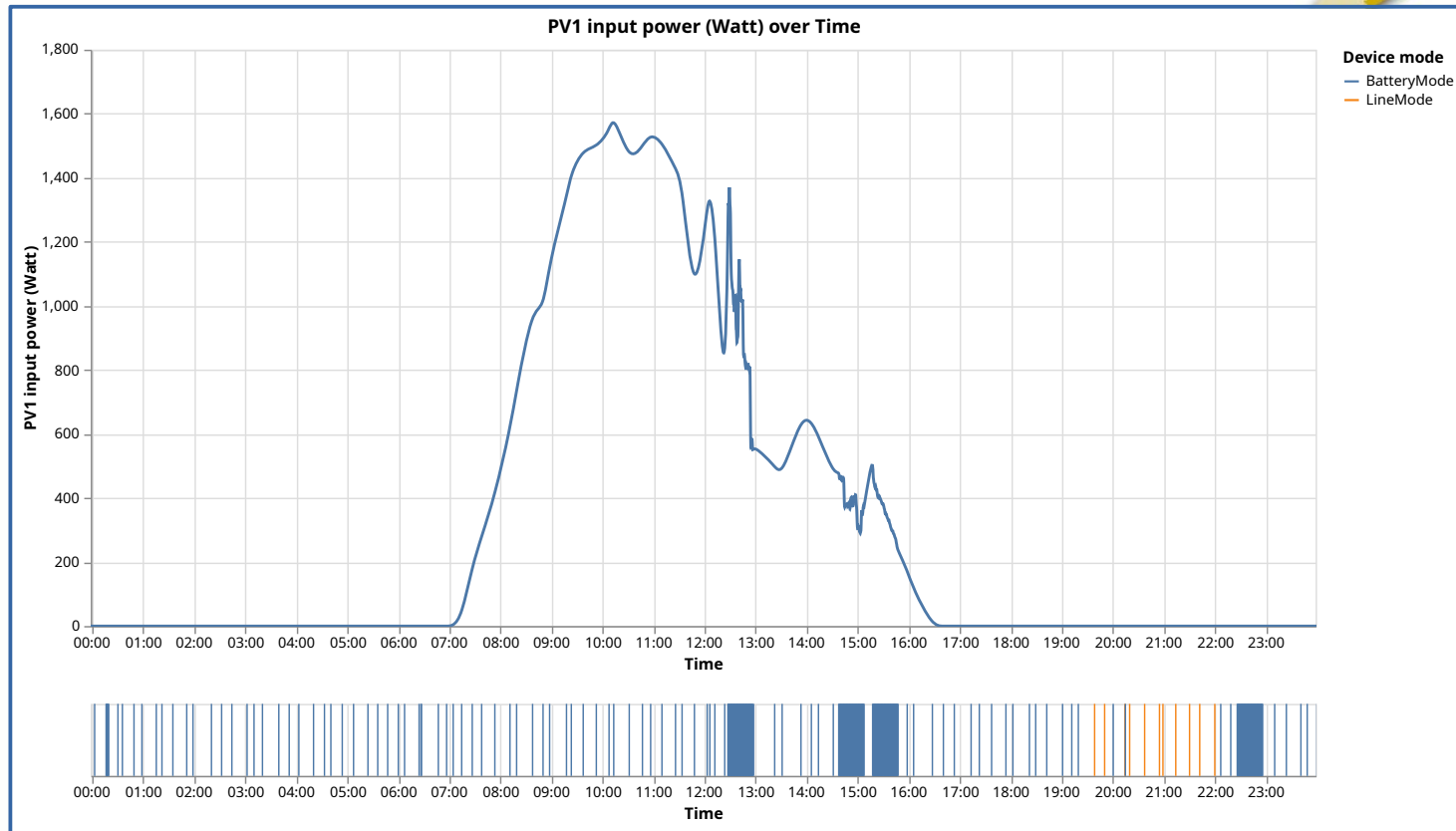
- Suppose you are a regular householder and want to use a PV system. You would have to choose the following:
 - How many PV panels to put?
 - How much battery capacity to put?
- For any choice, there would be either lot of power taken from your house grid line (i.e. too few PV panels, or too big a battery), or lot of excess power from PV panels (i.e. too many PV panels), or both (i.e. too small a battery).

Problem Statement



- Even with balanced parameters, this scenario may occur often:
 - If you need to turn on some big device (e.g. washing machine or air conditioner), the PV power may not cover your needs, so you take a lot of power from the grid (or just lack power).
 - If you don't need that and it's a sunny day, the battery will be fully charged and there would be lot of PV power unused.

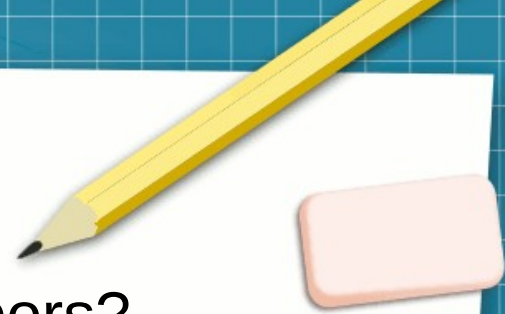
Problem Statement



PV input power for sunny day. Notice that after 13:00 there was lot of PV excess power.

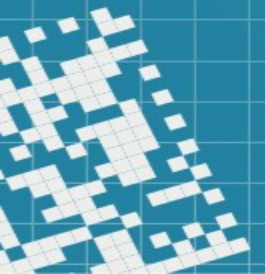
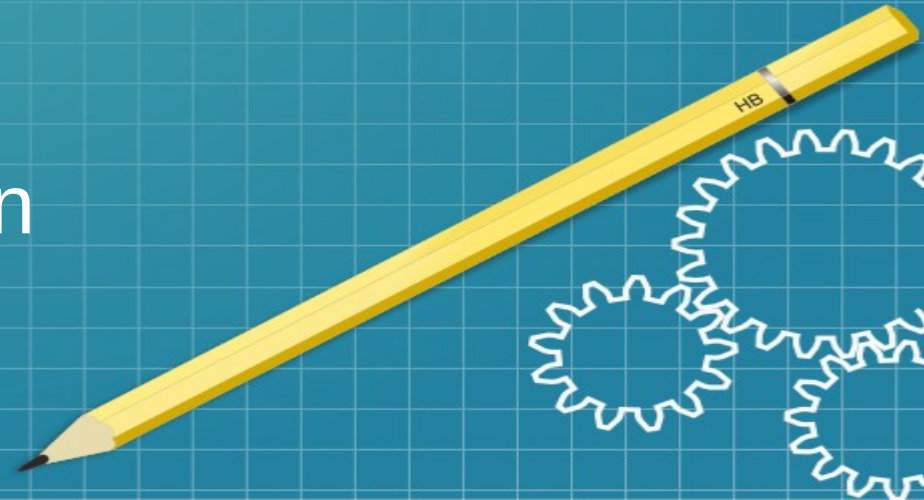
Problem Statement

- However, what if you collaborate with your neighbors?
- The excess of each house can be sent where there is shortage.
- This is the core idea of our project.



Project Overview

- Project Description
- Project Features
- Real System Design
- Prototype System Design



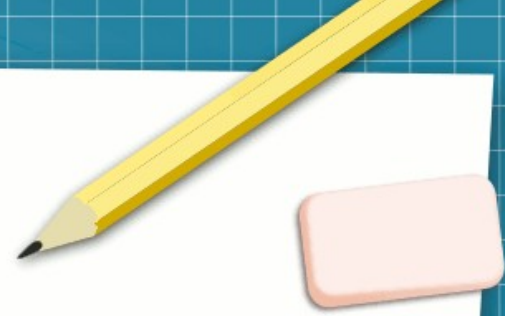
Project Description



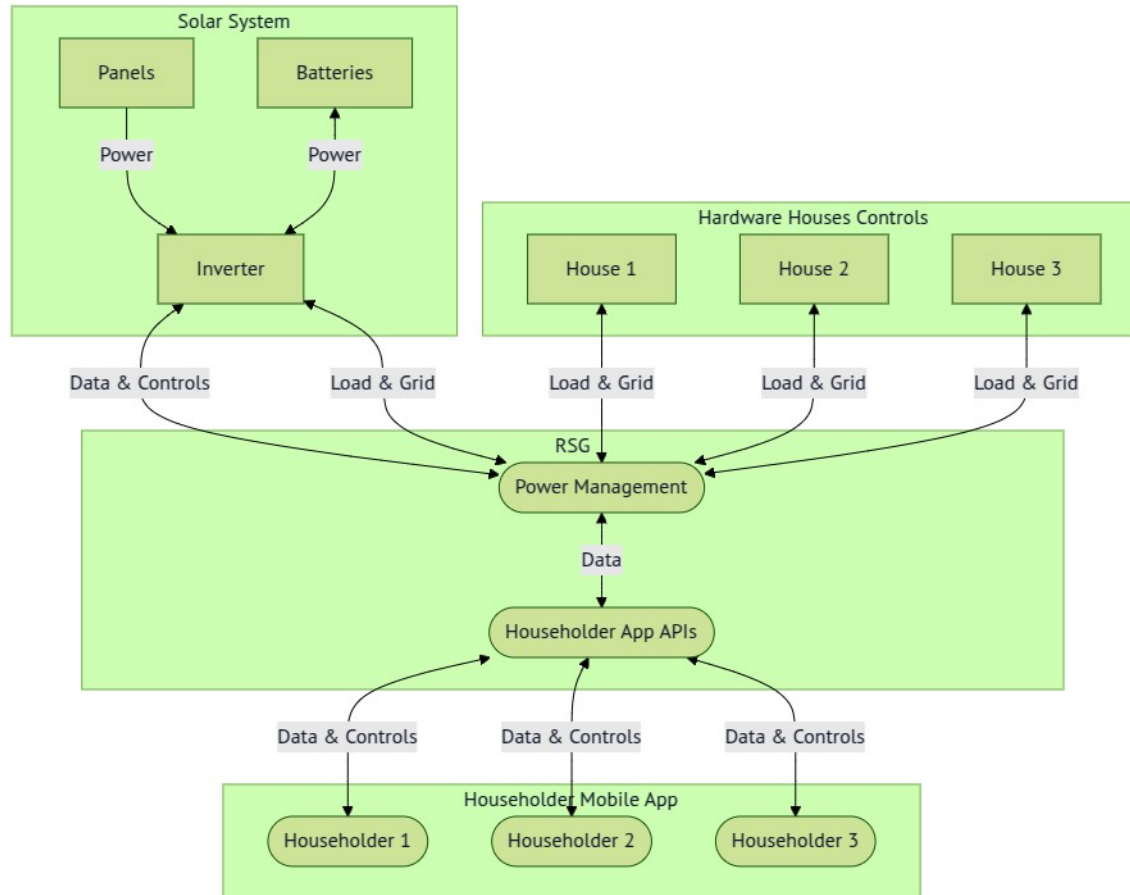
- RSG (Residential Smart Grid) is a grid system that links a large PV system for a residential building with each house in that building. This system controls the power flow between the PV system and each house load and grid lines, using smart techniques and methods.
- RSG aims to:
 - Minimize power taken from the houses' grid lines.
 - Minimize excess PV power.
 - Improve householders' experience.

Project Features

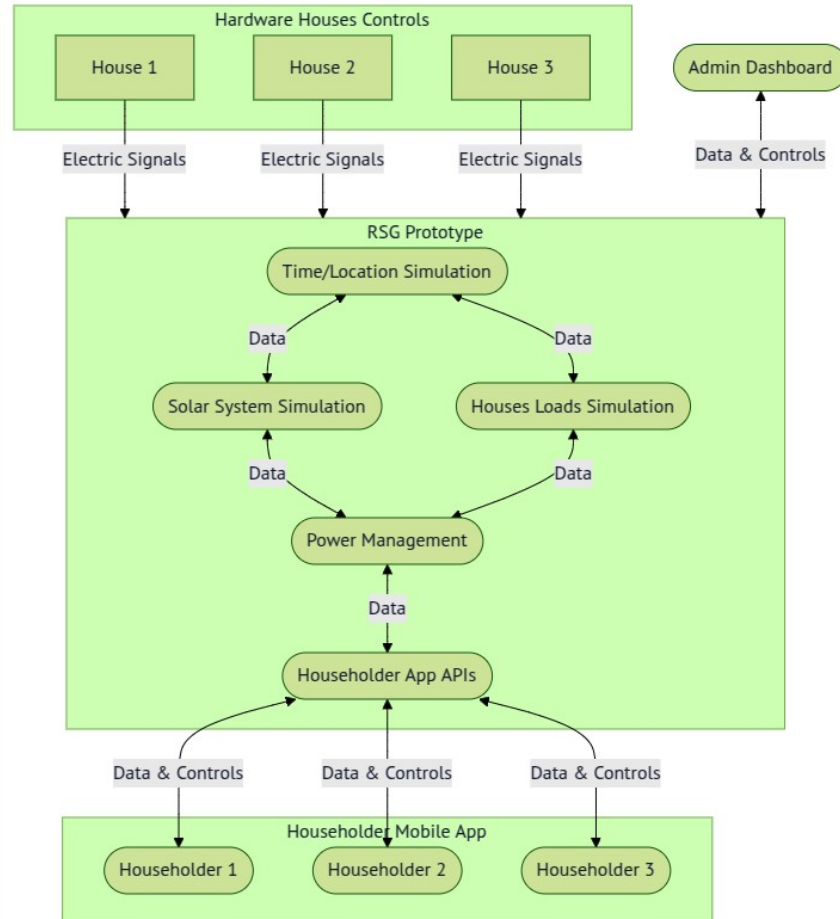
- Smart and transparent power management.
- Admin dashboard to browse and control the system.
- Householder mobile app to browse and control its system.



System Design



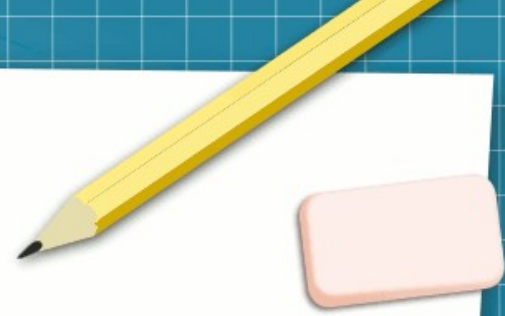
Prototype System Design



Current Progress & Plans

- Software Simulation
- Hardware Houses Controls
- Power Management
- Householder App APIs
- Householder Mobile App
- Admin Dashboard

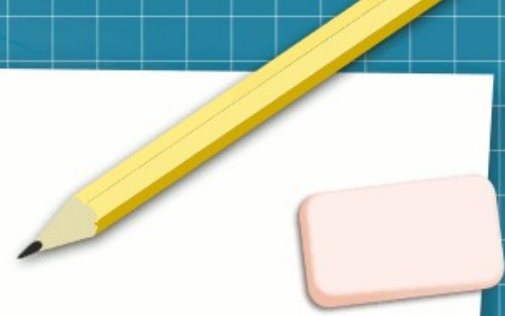
Software Simulation



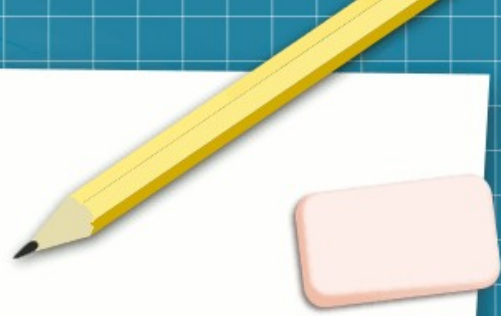
- **Time/Location Simulation:** Done.
- **Solar System Simulation:** A basic core with ideal settings.
- **Houses Loads Simulation:** A basic core that lacks a hardware implementation to track house controls.

Hardware Houses Controls

- Yet to be implemented.
- Plans:
 - Design house controls around the 40 pins that are available in total.
 - Implement and test the design.



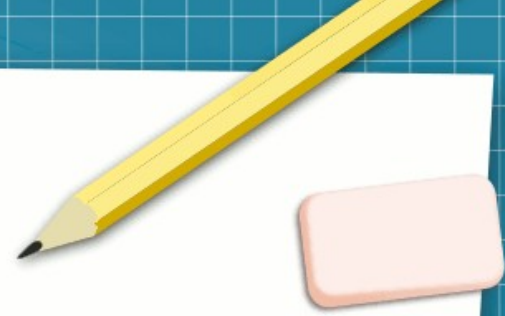
Power Management



- Basic static power management.
- Plans:
 - Survey the research available regarding power management using AI.
 - Implement the techniques derived from the researches.

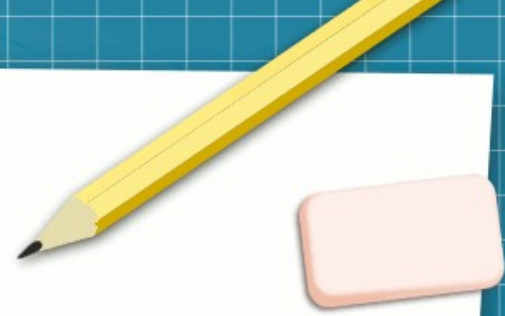
Householder App APIs

- Yet to be implemented.
- Plans:
 - Yet to be discussed.



Householder Mobile App

- Yet to be implemented.
- Plans:
 - Yet to be discussed.



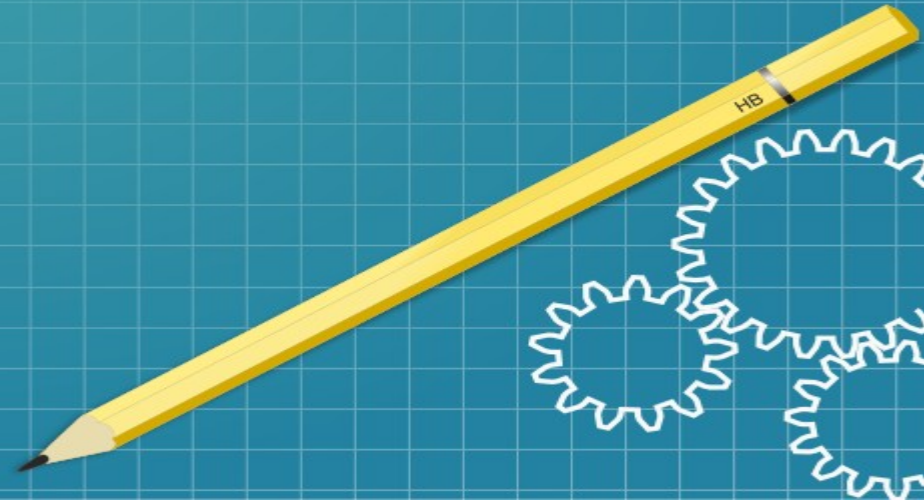
Admin Dashboard



- Implemented a TKinter app that runs with the RSG. What is implemented is as follows:
 - A View for the Houses Loads Simulation to control houses loads.
 - A View for Solar System Simulation.
 - A View for Power Management.
- Plans:
 - Split the dashboard from the RSG Prototype, since it could run in a separate process or even a separate computer (RSG Prototype can run on Raspberry Pi and Admin Dashboard on the admin's laptop, and they connect via network and exchange data through an APIs).

Notebooks

- Introduction to Notebooks
- Logs Notebook
- Graphs Notebook
- PV Notebook



Introduction to Notebooks



- Notebooks are tools that help in planning, visualization, and documentation. They are not a part of the core project.
- The main method we use to make and run notebooks is with Marimo; a Python tool for creating interactive notebooks that combine code, text, and visualizations.
- Unlike traditional notebooks (like Jupyter), Marimo automatically updates outputs when you change the code, making it a way to make dynamic, reactive documents.

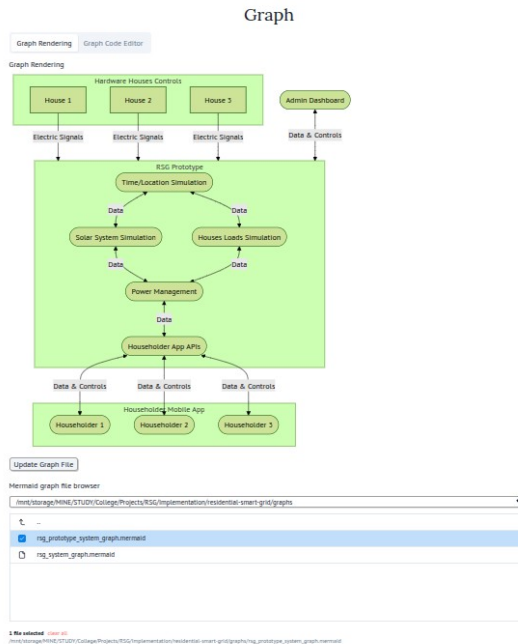
Logs Notebook

- This notebook visualizes RSG logs.



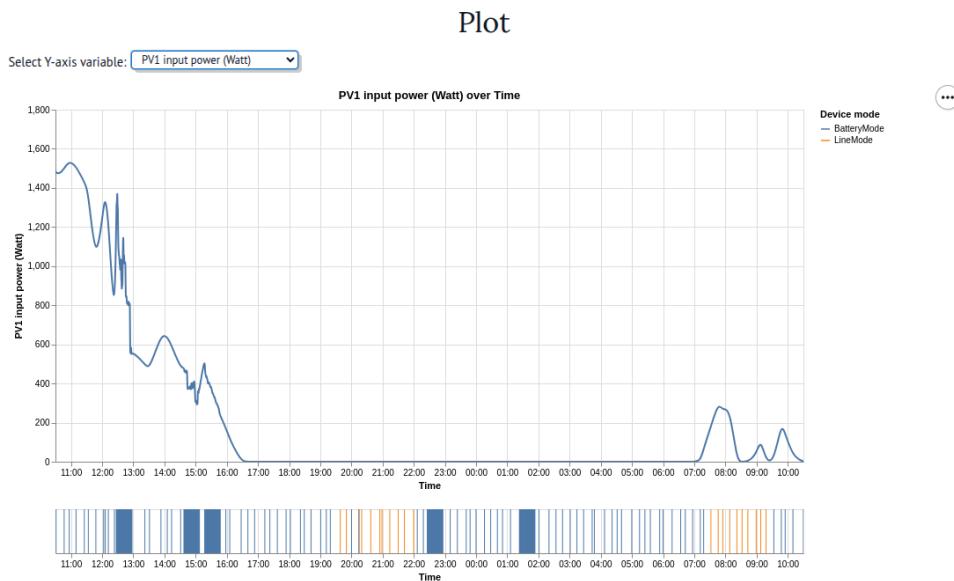
Graphs Notebook

- This notebook browses, edits, and renders Mermaid graphs.

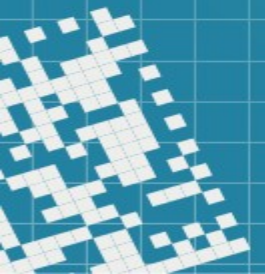
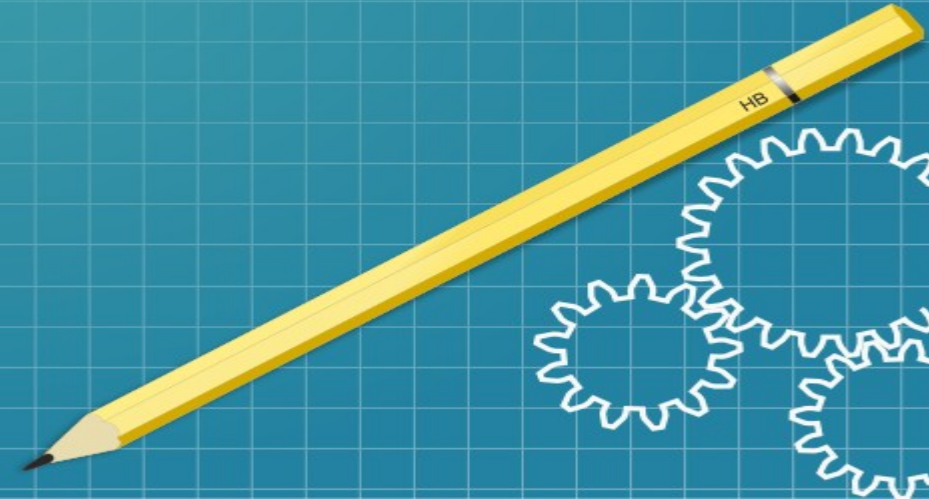


PV Notebook

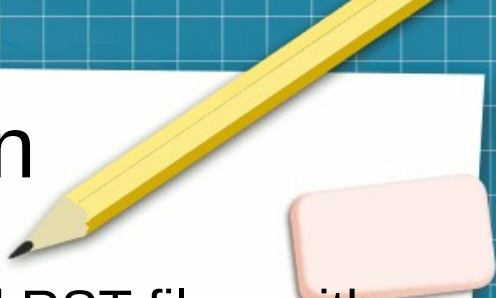
- This notebook visualizes PV logged data collected from a PV Inverter.



Documentation & Dissertation



Documentation & Dissertation



- For documentation and dissertation, we used Sphinx and RST files, with the following outline:
 - Chapter 01 Introduction
 - Chapter 02 Project Preview
 - Chapter 03 System Design
 - Chapter 04 Software and Hardware Simulation
 - Chapter 05 Power Management
 - Chapter 06 Householder App APIs and Mobile App
 - Chapter 07 Final Results
 - Chapter 08 Conclusions and Suggestions
 - Appendix 01 API Reference (Auto generated from the code)

Documentation & Dissertation

- With Sphinx, the documentation can be exported as a website (via HTML) and PDF (via LaTeX).

